

FUNDAMENTOS DE PROGRAMAÇÃO PARA INTERNET



Prof. Tiago Piperno Bonetti
bonetti@prof.unipar.br

O método toString ()

O método **toString()** retorna um número como uma string.

```
var numero = 50;  
numero.toString();  
(100).toString();  
(50 + 30).toString();
```

O método toFixed ()

toFixed() retorna uma string com o número escrito com um número **especificado** de **decimais**. Arredonda um número para um determinado número de dígitos.

```
var numero = 9.656;  
numero.toFixed(0);           // returns 10  
numero.toFixed(2);           // returns 9.66  
numero.toFixed(4);           // returns 9.6560  
numero.toFixed(6);           // returns 9.656000
```

O método toPrecision ()

toPrecision() retorna uma string, com um número escrito com um comprimento especificado:

```
var numero = 9.656;  
numero.toPrecision();           // returns 9.656  
numero.toPrecision(2);         // returns 9.7  
numero.toPrecision(4);         // returns 9.656  
numero.toPrecision(6);         // returns 9.65600
```

Convertendo Variáveis em Números

Existem 3 métodos JavaScript que podem ser usados para converter variáveis em números:

Método	Descrição
Number()	Retorna um número, convertido de seu argumento.
parseFloat()	Analisa seu argumento e retorna um número de ponto flutuante.
parseInt()	Analisa seu argumento e retorna um inteiro.

O Método Number ()

Number() pode ser usado para converter variáveis JavaScript em números:

```
Number(true);           // returns 1
Number(false);          // returns 0
Number("10");           // returns 10
Number(" 10");          // returns 10
Number("10.33");        // returns 10.33
Number("10,33");        // returns NaN
Number("10 33");        // returns NaN
Number("John");         // returns NaN
```

Se o número não puder ser convertido, NaN(Não é um número) será retornado.

O método `parseInt()`

`parseInt()` analisa uma string e retorna um número inteiro. Espaços são permitidos. Apenas o primeiro número é retornado.

```
parseInt("10");           // returns 10
parseInt("10.33");        // returns 10
parseInt("10 20 30");     // returns 10
parseInt("10 years");     // returns 10
parseInt("years 10");     // returns NaN
```

Método parseFloat ()

parseFloat() analisa uma string e retorna um número. Espaços são permitidos. Apenas o primeiro número é retornado:

```
parseFloat("10");           // returns 10
parseFloat("10.33");        // returns 10.33
parseFloat("10 20 30");     // returns 10
parseFloat("10 years");     // returns 10
parseFloat("years 10");     // returns NaN
```


Instrução switch

Use a **switch** para selecionar um dos muitos blocos de código a serem executados.

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block  
}
```

- ✓ A expressão switch é avaliada uma vez.
- ✓ O valor da expressão é comparado com os valores de cada caso.
- ✓ Se houver uma correspondência, o bloco de código associado é executado.
- ✓ Se não houver correspondência, o bloco de código padrão é executado.

Instrução switch

O **getDay()** retorna o dia da semana como um número entre 0 e 6.

(Domingo = 0, Segunda = 1, Terça = 2 ..)

Este exemplo usa o número do dia da semana para calcular o nome do dia da semana.

```
switch (new Date().getDay()) {  
    case 0:  
        day = "domingo";  
        break;  
    case 1:  
        day = "segunda-feira";  
        break;  
    case 2:  
        day = "terça-feira";  
        break;  
    case 3:  
        day = "quarta-feira";  
        break;  
    case 4:  
        day = "quinta-feira";  
        break;  
    case 5:  
        day = "sexta-feira";  
        break;  
    case 6:  
        day = "sabado";  
}
```

0 For Loop

○ **for loop** tem a seguinte sintaxe:

```
for (instrução 1; instrução 2; instrução 3) {  
    // bloco de código a ser executado  
}
```

A instrução 1 é executada (uma vez) antes da execução do bloco de código.

A instrução 2 define a condição para executar o bloco de código.

A instrução 3 é executada (sempre) após a execução do bloco de código.

0 For Loop

```
var text = "";  
var i;  
for (i = 0; i < 5; i++) {  
    text += "0 número é " + i + "<br>";  
}
```

O número é 0
O número é 1
O número é 2
O número é 3
O número é 4

0 For Loop

Loops são úteis, se você deseja executar o mesmo código repetidamente, cada vez com um valor diferente.

```
var cars = ["BMW", "Volvo", "VW", "Ford", "Fiat", "Audi"];
var text = "";
var i;
for (i = 0; i < cars.length; i++) {
    text += cars[i] + "<br>";
}
```

0 For Loop

Você pode iniciar muitos valores na afirmação 1 (separados por vírgula):

```
for (i = 0, len = cars.length, text = ""; i < len; i++) {  
    text += cars[i] + "<br>";  
}
```

Você pode omitir a instrução 1.

```
var i = 2;  
var len = cars.length;  
var text = "";  
for (; i < len; i++) {  
    text += cars[i] + "<br>";  
}
```

0 Loop For/In

A for/in percorre as propriedades de um objeto:

```
var txt = "";
var pessoa = {nome:"Tiago", sobrenome:"Bonetti", idade:25};
var atributo;
for (atributo in pessoa) {
    txt += pessoa[atributo] + " ";
}
document.getElementById("forin").innerHTML = txt;
```

0 For/Of Loop

Percorre os valores de objetos iteráveis. Permite que você faça um loop em estruturas de dados que são iteráveis, como Arrays, Strings e muito mais.

```
var carros = ["BMW", "Volvo", "VW"];  
for (carro of carros) {  
    document.write(carro + "<br>");  
}
```


While Loop

O whileloop percorre um bloco de código, desde que uma condição especificada seja verdadeira.

```
var text = "";  
var i = 0;  
while (i < 10) {  
    text += "<br>0 número é " + i;  
    i++;  
}
```

```
O número é 0  
O número é 1  
O número é 2  
O número é 3  
O número é 4  
O número é 5  
O número é 6  
O número é 7  
O número é 8  
O número é 9
```

O Loop Do/While

Este loop executará o bloco de código uma vez, antes de verificar se a condição é verdadeira e, em seguida, repetirá o loop enquanto a condição for verdadeira.

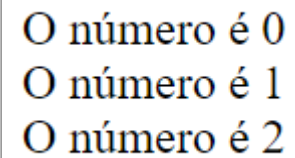
```
var text = ""
var i = 0;
do {
    text += "<br>O número é " + i;
    i++;
}
while (i < 10);
```

```
O número é 0
O número é 1
O número é 2
O número é 3
O número é 4
O número é 5
O número é 6
O número é 7
O número é 8
O número é 9
```

Break

A break quebra o loop e continua executando o código após o loop (se houver):

```
var text = "";  
var i;  
for (i = 0; i < 10; i++) {  
    if (i === 3) { break; }  
    text += "O número é " + i + "<br>";  
}
```



O número é 0
O número é 1
O número é 2

Continue

A continue interrompe uma iteração (no loop), se uma condição especificada ocorrer, e continua com a próxima iteração no loop.

```
var text = "";  
var i;  
for (i = 0; i < 10; i++) {  
    if (i === 3) { continue; }  
    text += "O número é " + i + "<br>";  
}
```

```
O número é 0  
O número é 1  
O número é 2  
O número é 4  
O número é 5  
O número é 6  
O número é 7  
O número é 8  
O número é 9
```

Referências

W3schools. <https://www.w3schools.com/js/default.asp>

Caelum. <https://www.caelum.com.br/apostila-html-css-javascript/>

Developer Mozilla:

[https://developer.mozilla.org/pt-](https://developer.mozilla.org/pt-BR/docs/Aprender/Getting_started_with_the_web/JavaScript_basico)

[BR/docs/Aprender/Getting_started_with_the_web/JavaScript_basico](https://developer.mozilla.org/pt-BR/docs/Aprender/Getting_started_with_the_web/JavaScript_basico)